

Dobot ROS2 Bridge

Isaac Sim Extension – Technische Dokumentation

OmniMazeRunnerExtensions
Tobias Küster

10. Juni 2026

Inhaltsverzeichnis

1	Übersicht	3
1.1	Ziele	3
1.2	Systemvoraussetzungen	3
2	Dateistruktur	3
3	Architektur	3
3.1	Datenfluss	4
4	Kinematisches Modell	4
4.1	Koordinatenkonventionen (pydobot)	4
4.2	Isaac-Sim-Gelenke und Einheitenstrategie	4
4.3	Berechnungsformeln (in Grad)	4
4.3.1	joint_1 und joint_2	4
4.3.2	joint_5 (Parallelogramm-Kompensation)	5
4.3.3	joint_6 (Endeffector-Vertikalisierung)	5
5	Module und Klassen	5
5.1	Lazy-Import-System	5
5.2	Klasse <code>DobotROSNode</code>	6
5.2.1	Fehlerbehandlung in <code>update_step()</code>	6
5.3	Klasse <code>DobotBridgeExtension</code>	7
6	Dynamic-Control-Kopplung	7
6.1	Initialisierungsproblem	7
6.2	Lazy-Initialisierung	7
6.3	Gelenkwinkel setzen	7
6.4	USD-Pfad der Artikulation	8
7	UI-Fenster	8
7.1	Docking	8
7.2	Layout-Übersicht	8
7.3	Connect-Button-Text	9
8	Konstanten (<code>constants.py</code>)	9

9	Installation und Inbetriebnahme	9
9.1	Erstinstallation	9
9.2	Betrieb	9
10	Bekannte Einschränkungen	9
11	PDF erzeugen	10

1 Übersicht

Die Extension `dobot_ros` stellt eine bidirektionale Kopplung zwischen einem realen **Dobot Magician**-Roboterarm und einem **NVIDIA Isaac Sim**-Simulationsmodell her.

1.1 Ziele

- **Digital Twin (Richtung 1):** Der echte Roboter bewegt sich und das Simulationsmodell folgt in Echtzeit.
- **Fernsteuerung (Richtung 2):** Über das UI-Panel werden Jog-Befehle, Vertikalbewegungen und der Sauger an den echten Roboter gesendet.
- **ROS2-Kompatibilität:** Gelenkzustände werden zusätzlich auf dem Topic `/joint_states` publiziert, damit externe ROS2-Knoten (z. B. MoveIt) die aktuelle Pose empfangen können.

1.2 Systemvoraussetzungen

Komponente	Version / Anforderung
NVIDIA Isaac Sim	5.1 oder neuer
Python	3.11 (Isaac-Sim-intern)
ROS2	Humble oder Jazzy (Windows-Bridge)
pydobot	≥ 1.3 (pip-installierbar)
pyserial	≥ 3.5 (Abhängigkeit von pydobot)
Dobot Magician	USB-Verbindung, COM-Port bekannt

2 Dateistruktur

Datei	Beschreibung
<code>extension.py</code>	Hauptlogik: ROS-Node, Kinematik, DC-Kopplung, UI
<code>constants.py</code>	Farb-, Pfad- und MQTT-Konstanten
<code>__init__.py</code>	Exportiert <code>DobotBridgeExtension</code>
<code>extension.toml</code>	Isaac-Sim-Manifest (Name, Version, Module)
<code>install_extension.py</code>	Kopiert Extension in den Benutzer-Erweiterungs-Pfad
<code>set_extension_path.ps1</code>	PowerShell-Skript zum Setzen des Extension-Pfads
<code>documentation.tex</code>	Diese Dokumentation (LaTeX-Quelle)
<code>build_docs.py</code>	Erstellt <code>documentation.pdf</code> aus dieser Datei

3 Architektur

Die Extension besteht aus zwei Hauptklassen:

DobotROSNode Kapselt die serielle USB-Verbindung zum echten Dobot (via `pydobot`) sowie den ROS2-Node und Publisher. Stellt `update_step()`, `set_suction()`, `move_vertical()` und `jog()` bereit.

DobotBridgeExtension

Subklasse von `omni.ext.IExt`. Verwaltet das Omniverse-UI-Fenster, die Isaac-Sim-Dynamic-Control-Kopplung und den Frame-Update-Loop.

3.1 Datenfluss

1. **Lesen:** `DobotROSNode.update_step()` ruft `device.pose()` auf und liest $(x, y, z, r, j_1, j_2, j_3, j_4)$ vom Dobot.
2. **Kinematik:** Die Rohwinkel werden in Isaac-Sim-kompatible Gelenkwinkel umgerechnet (siehe Abschnitt 4).
3. **ROS2-Publish:** Eine `JointState`-Nachricht wird auf `/joint_states` veröffentlicht.
4. **DC-API:** `DobotBridgeExtension._drive_joints()` setzt die Winkel direkt auf die Isaac-Sim-Artikulation.
5. **Dashboard:** Pose und Winkel werden im UI-Panel angezeigt.

4 Kinematisches Modell

4.1 Koordinatenkonventionen (pydobot)

Die pydobot-Bibliothek gibt absolute Weltwinkel in Grad zurück:

$$j_1 = \text{Basis-Rotation (Yaw)} \quad (1)$$

$$j_2 = \text{Oberarm-Winkel von Horizontal (absolut)} \quad (2)$$

$$j_3 = \text{Unterarm-Winkel von Horizontal (absolut)} \quad (3)$$

$$j_4 = \text{Endeffektor-Servoansteuerung (relativ)} \quad (4)$$

Durch den Parallelogramm-Mechanismus des Dobot Magician gilt näherungsweise $j_3 \approx 0$ (Unterarm bleibt horizontal in Weltkoordinaten), sofern der Roboter nicht extern manipuliert wird.

4.2 Isaac-Sim-Gelenke und Einheitenstrategie

Das Isaac-Sim-Modell (`/World/magician`) hat folgende Gelenke. USD-native `eREVOLUTE`-Joints speichern Winkel intern in **Grad**; die DC-API `set_dof_state` erwartet daher Grad. Ausnahme: `set_dof_position_target` (PhysX-Drive) erwartet Radiant.

Isaac-Sim-Gelenk	USD-Name	Bedeutung	Einheit (DC-API)
θ_1	<code>joint_1</code>	Basis-Rotation	Grad
θ_2	<code>joint_2</code>	Schultergelenk	Grad
θ_5	<code>joint_5</code>	Unterarm (Parallelogramm-Kompensation)	Grad
θ_6	<code>joint_6</code>	Endeffektor (Vertikalisierung)	Grad

4.3 Berechnungsformeln (in Grad)

Alle Formeln werden in Grad ausgeführt. Konvertierung nach Radiant erfolgt nur für den ROS2-Publisher und `set_dof_position_target`.

4.3.1 joint_1 und joint_2

Direkte Übernahme der pydobot-Werte (bereits in Grad):

$$\theta_1 = j_1 \quad [^\circ] \quad (5)$$

$$\theta_2 = j_2 \quad [^\circ] \quad (6)$$

4.3.2 joint_5 (Parallelogramm-Kompensation)

Da j_2 einen relativen Winkel in der Kette darstellt, j_3 aber einen absoluten Weltwinkel, muss θ_5 die Differenz ausgleichen, damit `link_5` (Unterarm) in der korrekten Weltlage bleibt:

$$\theta_5 = j_3 - j_2 \quad [^\circ] \quad (7)$$

Im Code:

```
1 pos_j5 = j3 - j2    # Grad
```

Listing 1: Berechnung joint_5 in Grad

4.3.3 joint_6 (Endeffector-Vertikalisierung)

Forderung: `link_6` soll unabhängig von der Armposition stets senkrecht zeigen (Saugnapf nach unten).

Herleitung: Die Summe aller Pitch-Rotationen entlang der kinematischen Kette muss in Weltkoordinaten 90 ergeben (Senkrechte, gemessen von Horizontal):

$$\theta_2 + \theta_5 + \theta_6 = 90 \quad (8)$$

Einsetzen von (7):

$$j_2 + (j_3 - j_2) + \theta_6 = 90 \quad (9)$$

$$j_3 + \theta_6 = 90 \quad (10)$$

Der j_2 -Term kürzt sich heraus. Damit:

$$\boxed{\theta_6 = 90 - j_3 - j_4} \quad (11)$$

j_4 wird subtrahiert, da ein positiver Servo-Wert eine Rotation entgegen der Vertikalisierungsrichtung bewirkt.

Wichtiger Hinweis: j_2 darf *nicht* in Gleichung (11) auftreten. Ein früherer Fehler (`pos_j6 = j2 + j3 - j4`) ließ θ_6 mit der Schulterposition mitlaufen, wodurch der Endeffector bei gestrecktem Arm nach oben kippte. Ein weiterer Fehler mischte irrtümlich $\frac{\pi}{2}$ (Radiant) mit Grad-Werten — jetzt ist die Formel konsequent in Grad formuliert.

Im Code:

```
1 # Kette: j2 + (j3 - j2) + j6 = 90 => j6 = 90 - j3 - j4
2 # j2 darf hier NICHT stehen
3 pos_j6 = 90.0 - j3 - j4    # Grad
```

Listing 2: Korrekte Berechnung joint_6 in Grad

5 Module und Klassen

5.1 Lazy-Import-System

ROS2 (`rclpy`) und `pydobot` werden nicht beim Extension-Start importiert, sondern erst beim ersten Connect-Klick. Dadurch startet die Extension auch in Umgebungen ohne ROS2.

```

1 _rclpy = None      # Cache: rclpy-Modul
2 _Node = None       # Cache: rclpy.node.Node
3 _JointState = None # Cache: sensor_msgs.msg.JointState
4 _Dobot = None      # Cache: pydobot.Dobot
5
6 def _try_import_ros():
7     global _rclpy, _Node, _JointState
8     if _rclpy is not None:
9         return True # bereits gecacht
10    try:
11        import rclpy
12        from rclpy.node import Node
13        from sensor_msgs.msg import JointState
14    except Exception:
15        return False
16    _rclpy, _Node, _JointState = rclpy, Node, JointState
17    return True

```

Listing 3: Lazy-Import-Mechanismus

5.2 Klasse DobotROSNode

Methode	Beschreibung
<code>__init__(com_port)</code>	Öffnet COM-Port, initialisiert ROS2-Node und Publisher auf <code>/joint_states</code> .
<code>update_step()</code>	Liest Pose vom Dobot, berechnet Gelenkwinkel nach Abschnitt 4, publiziert JointState, gibt (<code>pose_dict</code> , <code>joints_list</code>) zurück.
<code>set_suction(active)</code>	Schaltet Vakuumgreifer; probiert mehrere pydobot-API-Namen für Versionskompatibilität.
<code>move_vertical(delta_mm)</code>	Liest aktuelle Pose, addiert <code>delta_mm</code> auf Z, sendet <code>move_to</code> -Befehl.
<code>jog(dx, dy)</code>	Wie <code>move_vertical</code> , aber in X-Y-Ebene.
<code>shutdown()</code>	Schließt COM-Port und zerstört ROS2-Node.

5.2.1 Fehlerbehandlung in `update_step()`

Bei Kommunikationsunterbrechungen (USB-Ausfall, Timeout) werden die letzten gültigen Gelenkwinkel aus dem Puffer `last_valid_positions` verwendet. So springt das Simulationsmodell nicht auf Nullstellung zurück.

5.3 Klasse DobotBridgeExtension

Methode	Beschreibung
<code>on_startup(ext_id)</code>	Erstellt UI-Fenster, initialisiert Zustandsvariablen, startet Docking-Task.
<code>_dock_window_deferred()</code>	Async-Coroutine: wartet 2 Frames, dockt Fenster neben das Property-Panel.
<code>_init_articulation(path, silent)</code>	Bindet die Isaac-Sim-DC-API an die Roboter-Artikulation. Wird solange wiederholt, bis Physik läuft.
<code>_drive_joints(joints_rad)</code>	Setzt Gelenkwinkel über <code>set_dof_state</code> und <code>set_dof_position_target</code> auf das Modell.
<code>_on_update(event)</code>	Frame-Callback: DC-Retry, Dobot-Polling, Modellkopplung, Dashboard-Update.
<code>_on_connect_clicked()</code>	Startet Verbindung, markiert DC-Init als ausstehend.
<code>_on_disconnect_clicked()</code>	Trennt Verbindung, setzt UI-Zustand zurück.
<code>_cleanup()</code>	Gibt alle Ressourcen frei (Subscription, DC, ROS2-Node).
<code>on_shutdown()</code>	Cancel Dock-Task, ruft <code>_cleanup()</code> , zerstört Fenster.

6 Dynamic-Control-Kopplung

6.1 Initialisierungsproblem

Die Isaac-Sim-Dynamic-Control-API (`omni.isaac.dynamic_control`) benötigt eine *laufende* Physik-Simulation. Ein Aufruf von `get_articulation()` vor dem Drücken von *Play* liefert `INVALID_HANDLE` und erzeugt eine Warning im Log.

6.2 Lazy-Initialisierung

Um Log-Spam zu vermeiden, wird `_init_articulation()` nur dann aufgerufen, wenn `omni.timeline` meldet, dass die Simulation läuft:

```

1 if self._art_init_pending and self._dc is None:
2     import omni.timeline
3     if omni.timeline.get_timeline_interface().is_playing():
4         self._init_articulation(robot_path, silent=True)
5         if self._dc is not None:
6             self._art_init_pending = False

```

Listing 4: Lazy DC-Initialisierung in `_on_update()`

6.3 Gelenkwinkel setzen

Pro Gelenk werden zwei DC-Aufrufe verwendet, um beide möglichen Konfigurationen abzudecken:

```

1 # Formeln liefern Grad; DC-API erwartet Radiant fuer beide Aufrufe
2 rad = math.radians(float(angle_deg))
3
4 # 1) Kinematischen Zustand direkt setzen (kein PD-Drive noetig)
5 state = self._dc_mod.DofState()
6 state.pos = rad    # Radiant
7 state.vel = 0.0

```

```

8 self._dc.set_dof_state(dof_handle, state, self._dc_mod.STATE_POS)
9
10 # 2) PhysX-Drive-Target setzen (Bereich  $[-2\pi, 2\pi]$  Pflicht)
11 self._dc.set_dof_position_target(dof_handle, rad)

```

Listing 5: Doppelter DC-Aufruf pro Gelenk

Einheitenstrategie:

- Interne Berechnungen (`update_step`) in **Grad** — einfachere Formeln, direkte Übernahme der pydobot-Werte.
- Beide DC-API-Aufrufe erwarten **Radian**; Konvertierung erfolgt in `_drive_joints` mit `math.radians()`.
- `set_dof_position_target` erzwingt $[-2\pi, 2\pi]$:

`PxArticulationJointReducedCoordinate::setDriveTarget()` only supports target angle in range $[-2\pi, 2\pi]$

6.4 USD-Pfad der Artikulation

Die `ArticulationRootAPI` muss auf dem Prim liegen, an dem `get_articulation()` sucht. Beim Dobot-Magician-Import liegt sie auf `/World/magician/base_link`, *nicht* auf `/World/magician`.

7 UI-Fenster

7.1 Docking

Das Fenster dockt automatisch als Tab neben das Isaac-Sim-*Property*-Panel:

```

1 async def _dock_window_deferred(self):
2     await omni.kit.app.get_app().next_update_async()
3     await omni.kit.app.get_app().next_update_async()
4     prop = ui.Workspace.get_window("Property")
5     if prop:
6         self._window.dock_in(prop, ui.DockPosition.SAME)

```

Listing 6: Deferred Docking

Zwei Frames Verzögerung sind nötig, da das Fenster erst nach dem ersten Render-Zyklus im Workspace-Index auftaucht.

Der `asyncio.Task` wird in `on_shutdown()` gecancelt, um die Warnung „*extension object is still alive*“ zu vermeiden.

7.2 Layout-Übersicht

UI-Element	Funktion
Titel-Label	Beschriftung
COM-Port-Feld + Install	Serielle Schnittstelle; Paketinstallation
Robot-USD-Feld	USD-Pfad der Artikulation in der Szene
Connect / Disconnect	Verbindungsaufbau/-trennung
Suction ON/OFF	Vakuumgreifer
Up / Down 10 mm	Vertikalbewegung ± 10 mm
X $\pm$ / Y $\pm$ 5 mm	Horizontaler JOG ± 5 mm
Status-Label	Aktueller Betriebsstatus
Pose-Label	Kartesische Position (x, y, z, r)
Joints-Label	Gelenkwinkel in Grad

7.3 Connect-Button-Text

Der *Connect*-Button zeigt nach erfolgreicher Verbindung dauerhaft **Connected** (COMx) an, damit der verbundene Port auch dann sichtbar bleibt, wenn die Status-Zeile durch spätere Nachrichten überschrieben wird.

8 Konstanten (constants.py)

Konstante	Bedeutung
ROBOT_USD_PATH	Standard-USD-Pfad der Artikulations-Root (/World/magician/base_link)
CLR_*	ARGB-Farbwerte für das UI-Styling
MQTT_BROKER, MQTT_PORT	Verbindungsparameter für den MQTT-Broker
API_URL_GET, API_URL_SET, API_KEY	REST-API-Endpunkte für Datenbankzugriff
SUCTION_*, PLACE_*	USD-Pfade für den Sauggreifer und das Pick-&-Place-Setup
DECKEL_START_LOCAL_POS/ROT	Startposition des Deckels im Magazin

9 Installation und Inbetriebnahme

9.1 Erstinstallation

1. **Extension einbinden:** In Isaac Sim unter *Window* → *Extensions* → *gear-Icon* den Pfad `C:\Bachelorarbeit\OmniMazeRunnerExtensions` als Extension-Suchpfad hinzufügen.
2. **Extension aktivieren:** `dobot_ros` in der Extension-Liste suchen und aktivieren.
3. **Pakete installieren:** Auf *Install packages* klicken, um `pydobot` und `pyserial` zu installieren. Danach Extension deaktivieren und neu aktivieren.

9.2 Betrieb

1. Isaac Sim starten und eine Szene mit dem Dobot-Magician-Modell laden (/World/magician).
2. ArticulationRootAPI auf `base_link` prüfen (Property-Panel: *Physics* → *Articulation Root*).
3. Extension aktivieren; das Panel dockt neben *Property*.
4. COM-Port eingeben (Standard: COM3) und **Connect** klicken. Status: *Warte auf Play...*
5. **Play** drücken. Status: *Modell verbunden: 4/4 DOFs*.
6. Das Simulationsmodell folgt dem echten Roboter in Echtzeit.

10 Bekannte Einschränkungen

- ROS2 muss unter Windows als Bridge (z. B. über WSL2 oder ROS2 for Windows) verfügbar sein, damit `rclpy` importiert werden kann.
- Die DC-API gibt eine Warning aus (*Failed to find articulation*), wenn `get_articulation()` vor dem Start der Simulation aufgerufen wird. Dies ist mit der Lazy-Initialisierung auf das erste Play-Ereignis beschränkt.

- Die Sauger-Methoden in pydobot variieren zwischen Versionen; die Fallback-Liste in `set_suction()` deckt die bekannten Varianten ab.
- Physikalisch ungültige Winkel (außerhalb $[-2\pi, 2\pi]$) bei der DC-API führen zu einem PhysX-Fehler. Die Winkel aus `update_step()` liegen immer in diesem Bereich.

11 PDF erzeugen

Im Extension-Ordner ausführen:

```
python build_docs.py
```

Das erzeugt `documentation.pdf` aus dieser Datei.